DaveB's Freecell: A "Thin Man" Style Video Script

A script for a short explainer video about this project, styled as a pastiche of the 1934 film *The Thin Man*: **Nick Charles** (the technical half of the pair, dry wit, explains the project) and **Nora Charles** (the curious half, asks the questions), with their wire fox terrier **Asta** as a silent, recurring visual gag. It's a script plus *generation directions* — not generated media itself. You supply a photo and a voice recording; you pick the actual voice-cloning and photo-animation tools (see "Generation pipeline" below) and run them yourself, the same way the original plain cut at the end of this file was meant to be fed into a TTS/podcast tool of your choice.

Cast and likeness plan

Character	Likeness source	Voice source
Nick	An actual photo of **Dave Boyd**, animated	**Voice-cloned from a Dave Boyd recording** (see "Generation pipeline" step 1 below) — Nick speaks in Dave's own synthesized voice
Nora	An actual photo of Deb Boyd, animated	An off-the-shelf TTS voice (warm, dry-witted delivery) — there's no Nora voice recording to clone from, so this is the one asymmetry in fidelity unless you record a second voice sample
Asta	Stock footage or a brief text-to-video/photo-animation clip of a wire fox terrier	None (silent reaction shots only)

A note on likeness rights: Nick's likeness and voice are your own (Dave Boyd), and Nora's likeness is Dave's wife (Deb Boyd). This is styled as an *homage/pastiche* to a classic film archetype, not a claim to be footage of the real William Powell or Myrna Loy — keep any credits/description on the finished video honest about that (e.g., "styled after the Thin Man films," not "starring William Powell").

Generation pipeline

This is tool-agnostic on purpose — pick whichever voice-cloning and photo-animation services you're comfortable with; the steps are the same shape regardless of vendor:

1. **Record the voice sample.** Read the paragraph marked **"RECORD THIS FOR VOICE CLONING"** below aloud, once cleanly, in a quiet room — most voice-cloning tools want 30-60+ seconds of a single, uninterrupted voice with minimal background noise. This becomes the reference clip for cloning Nick's voice.
2. **Clone the voice.** Feed that recording into a voice-cloning service to produce a synthetic voice model of Dave Boyd's voice, then synthesize *all* of Nick's lines below (not just the sample paragraph) through that cloned voice.
3. **Pick or generate Nora's voice.** A stock TTS voice with a warm, knowing delivery works fine; there's no lookalike voice recording to clone from here.
4. **Animate the two stills.** Feed the Dave Boyd photo plus Nick's synthesized audio track into a "talking photo" / photo-to-video lip-sync tool to animate Nick; do the same with the Deb Boyd still image plus an audio track and output a lip-synced talking-head clip.
5. **Style pass (optional but recommended).** Run both talking-head clips through a black-and-white / film-grain style filter to match the 1930s *Thin Man* look, rather than leaving them in plain color.
6. **Get Asta.** Source a short stock clip (or a quick text-to-video generation) of a wire fox terrier for the `[ASTA REACT]` cutaway beats marked in the script.
7. **Assemble.** Cut the two talking-head tracks together per the scene directions below, inserting the Asta cutaways and the B-roll screenshots (the actual game/dashboard) at the marked points, in any ordinary video editor.

Scene setting

A 1930s apartment study: a card table with a FreeCell deal laid out, a decanter and two glasses, Asta curled up by the table leg. Static talking-head coverage of NICK and NORA is enough for the whole runtime; cut to B-roll (the terminal, the desktop GUI, the web dashboard) at the marked points instead of trying to stage anything more elaborate.

[NICK, seated at the card table, gives the spread a sardonic once-over]

NORA: Darling, everyone's asking — why on earth does a game of solitaire need its own documentary?

NICK: Because, angel, this isn't *just* solitaire. It's a FreeCell game built in Rust, test-first, that grew into three different user interfaces, a full statistics and self-analysis system, a solver built from scratch, and a web dashboard written in JavaScript. Every last piece of it shares one game engine. No duplicated rules anywhere — I checked.

NORA: You always did like things tidy. Where does one even start with a thing like that?

> RECORD THIS FOR VOICE CLONING (read this paragraph aloud once, cleanly, for the voice sample):

NICK: With the engine, dear — that's the heart of the whole affair. It's a pure Rust library with no I/O at all: no printing, no file access, nothing but data and functions. The board — the cascades, the free cells, the foundations — is one plain structure called `GameState`. And here's the interesting part: the whole thing is built Redux-style, the way modern JavaScript applications manage state. Every way the game can change — moving a card, undoing a move, auto-playing to the foundations, dealing a fresh hand — is just one value in an action list. One function takes a state and an action and hands you back a new state. No mutation, no surprises, no side effects to speak of.

[ASTA REACT: a single, unimpressed glance up at Nick, then back to sleep]

NORA: Redux. In a card game. You *do* know how to charm a girl.

NICK: It pays off, though. Undo and redo fall right out of it — just stacks of past and future states. And a whole game is *completely* described by two things: the seed it was dealt from, and the list of moves played. Nothing else. So there's a function that takes a seed and a move list and rebuilds the exact game from scratch. Every time this thing detects a win, it actually replays the entire game and confirms it reproduces that exact win — a running proof, not a test that quietly rusts.

NORA: And that's the trick behind all these different faces it wears?

NICK: Precisely. A terminal version, a desktop application, a browser version compiled to WebAssembly, and — as of recently — an Android build. Four of them, and not one knows a single rule of FreeCell on its own. The supermove math, the alternating colors, all of it lives in exactly one place.

[CUT TO: screenshot/recording of the terminal CLI dealing a hand]

NORA: Let's talk statistics, Nicky. I hear this thing keeps rather better records than you do.

NICK: *(a dry look)* There's a `stats` module — a plain list of game results plus some arithmetic over it: win percentage, current streak, longest winning and losing runs. The clever bit is how it *gets* that data: it simply subscribes to the Redux store. Every time a move goes through successfully, the stats module hears about it. It needs no special hook into the rules at all.

NORA: So any new interface gets the bookkeeping for free?

NICK: Proven three times over, my dear — all three interfaces share it. It even catches the case of someone simply walking away mid-hand — closing the window, cutting the lights — and still marks it down as a loss. No slipping out unnoticed.

NORA: Now — the part I actually want to hear about. You mentioned a *solver*. Are you telling me the machine plays the hand for you?

NICK: It tells you whether a position can *still* be won, and if so, shows you the whole winning line. A depth-first search with a transposition table — it remembers every position it's already explored, so it never wastes a moment re-exploring the same board reached by a different order of moves. And there's a classic FreeCell trick built in: a "safe autoplay" rule that sends a card home the instant doing so can never possibly hurt you later.

NORA: How fast is "fast," exactly? I understood this game to be fiendish.

NICK: It is. There's a famous hand — number 11982 in the old Microsoft numbering — proven mathematically unwinnable. This solver

proves that in about seven and a half seconds, on a slow build no less. Most hands resolve considerably quicker.

[ASTA REACT: ears perk up at "fiendish," then settle back down]

NORA: And that little "hint" button — is that the same machinery?

NICK: The same machinery, on a shorter leash. Hints use a much smaller search budget, tuned down so it stays responsive during actual play instead of freezing everything for several seconds. If that smaller budget comes up empty, it says so plainly — "no hint available" — rather than pretending the hand is lost.

NORA: That honesty seems almost out of character for you.

NICK: *(raising a glass)* Wound me. But it matters — the solver gives three honest answers, not two: solvable, unsolvable, or *unknown*, meaning it simply didn't search deep enough to be certain. Every piece of code touching it has to handle "unknown" as its own real case. Get that wrong, and you'd tell a player their hand is dead when it might still be very much alive.

NORA: You mentioned some sort of report card, too.

NICK: One of my favorite bits, if I'm honest. When a hand ends — or even when it's simply abandoned — there's a function that grades it: not merely "you lost," but the *exact* move where the position stopped being winnable.

NORA: However do you work that out without solving every single position along the way?

NICK: Here's the elegant part, angel. Along any one real game, once a position turns unwinnable, it *stays* unwinnable for the rest of that game — it can never flip back. Which means you can binary-search for the exact turning point instead of checking every move in sequence. On a sixty-move game, that's the difference between roughly sixty checks and about six.

NORA: *(a small, genuine smile)* That's rather elegant, for a card game.

NICK: It's the sort of thing that only turns up when you actually sit and think about what "still winnable" means over a real sequence of moves, rather than brute-forcing the question.

[CUT TO: screen recording of the desktop GUI's Hint/Report buttons in use]

NORA: Let's talk pictures — I understand there are actual charts involved these days.

NICK: Two tracks, quite deliberately. One lives in the application itself — charts drawn with a Rust charting library, right inside the desktop and browser builds: a win-rate trend over time, and a breakdown of how many moves your games tend to run, split by wins and losses.

NORA: And the second?

NICK: An entirely separate web dashboard, written in plain JavaScript with D3 — no build tools, no installation step, just static files sitting on a server. It reads the very same exported data the Rust side produces. Hover a point on the win-rate chart and it'll tell you which hand it was; click it, and it reopens that exact numbered deal for a rematch.

NORA: Why bother building the same chart twice, in two different languages, darling?

NICK: Because of one strict house rule: *all* the arithmetic stays in Rust. The data format is versioned and tested — changing its shape is treated like a real breaking change, never a casual edit. JavaScript is only ever permitted to *render* what's already been calculated, never to calculate anything new itself. Which means either one of those two renderers could be torn out and rebuilt in an entirely different language tomorrow, without touching a line of the actual game logic.

NORA: One more thing — you said clicking a chart point reopens "that exact numbered deal." Is that the very game I played, replayed?

NICK: *(honest, no dodge)* No, and it's worth being precise about it. What's kept on record is only the deal number, whether it was won, and how many moves it took — not the full move-by-move log. So a "replay" link deals that same numbered hand fresh, rather than replaying your exact recorded moves. Since the deal number alone determines the starting layout, it's still the identical puzzle — just not a literal instant-replay of your particular playthrough.

[CUT TO: browser recording of the web dashboard — hover, then click, a chart point]

NORA: So — soup to nuts, darling, what does the finished article actually look like?

NICK: A text interface, for the purists. A full terminal application with mouse support and properly colored suits. A desktop application with hand-drawn vector suit symbols — not font characters dressed up to look like cards. A browser version compiled straight to WebAssembly. And, most recently, a locally-built Android package, using that very same desktop code, simply dressed for a telephone.

NORA: All from the one engine.

NICK: All from the one engine. Continuous testing runs formatting checks, linting, and the full test suite on every single change, plus coverage reporting. A release process packages binaries for three operating systems and posts them the moment a version is tagged. And the dependency-watching runs on its own, merging the small, safe updates automatically once the tests pass.

NORA: That's rather a lot of engineering for a game of solitaire, Nicky.

NICK: That's rather the point, my dear. FreeCell is simple enough to hold entirely in one's head, which makes it an ideal place to practice good architecture without the subject matter getting in the way. The solver, the statistics, the two picture-drawing tracks — each one small enough to reason about on its own. Which is exactly why it was possible to keep building on top of it without anything coming apart at the seams.

NORA: Wherever does one go from here?

NICK: There's a set of notes alongside this script on FreeCell strategy itself — the patterns a solver, or a rather good player, falls into without necessarily naming them. Beyond that, the natural next steps are things like solver-backed grading across an entire history of games, or new interfaces plugging into the same engine the same effortless way these four already do.

NORA: Well. That's DaveB's Freecell, everyone. The same fifty-two cards you've always known —

NICK: — with rather more going on underneath than you'd expect.

[ASTA REACT: sits up, tail wagging, as if on cue]

NORA: *(to Asta)* Every column laid out and dealt, to prove it.

[FADE OUT]

Appendix: plain two-host cut

If you'd rather generate a plain audio-only "podcast" (feeding this into a NotebookLM-style audio-overview tool, for instance) instead of a styled video, strip the character names/production directions above and read it as two generic hosts, **A** (curious) and **B** (technical) — line-for-line the same content, just delivered straight rather than in character. The dialogue above works either way; the "Cast and likeness plan" and "Generation pipeline" sections only apply to the styled video version.

(End of script.)